SYMBIOSIS INSTITUTE OF TECHNOLOGY (SIT)
Constituent of Symbiosis International (Deemed University), Pune
(Established under Section 3 of the UGC Act of 1956 vide notification number F-9-12/2001-U-3 of the Government of india)
॥वसुधैव कुटुम्बकम्॥ Re-Accredited by NAAC with 'A++' Grade

Subject: Programming with Java

ASSIGNMENT No :3

TITLE : Develop a Java application demonstrating method overloading and the use of static variables, methods in class design.

Name: Arpan Gupta	Class: CSE-A2
PRN: 25070122046	Batch: 2025-2029

Problem Statement

Develop a Java application demonstrating method overloading and the use of static variables, methods in class design.

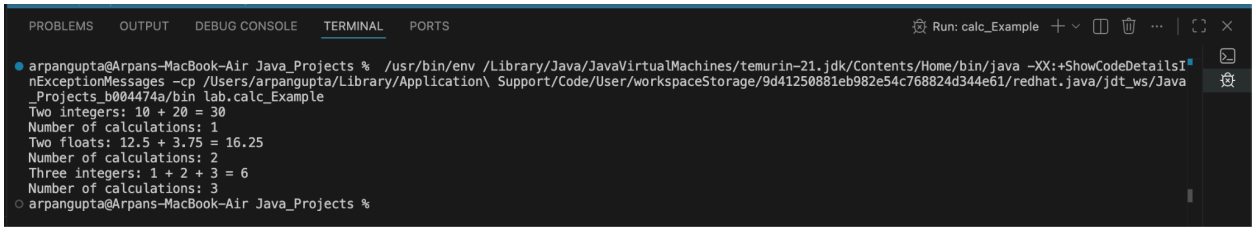
Learning Objectives

To implement method overloading and use static variables and methods in Java programs.

Theory

Method overloading allows multiple methods to have the same name with different parameter lists, enabling compile-time polymorphism. The static keyword is used for variables and methods that belong to the class instead of individual objects. Static members share common data among all objects and reduce memory usage. These concepts improve code flexibility, readability, and efficient resource management.

Outcome



Conclusion

The program successfully demonstrated method overloading by using methods with the same name but different parameters. This feature enables compile-time polymorphism and makes Java programs more efficient and organized.

Exercise

1. Develop a **Calculator program** using overloaded methods for addition of integers and decimals. Use a static variable to count calculations.

calc_Example.java

Something > src > lab > calc_Example.java > calc_Example > main(String[])

```java
package lab;

class Calc {
    int calc_count = 0;

    int add(int a, int b) {
        calc_count++;
        return a + b;
    }

    float add(float x, float y) {
        calc_count++;
        return x + y;
    }

    int add(int l, int m, int n) {
        calc_count++;
        return l + m + n;
    }
}

public class calc_Example {
    Run | Debug
    public static void main(String args[]) {
        Calc c = new Calc();

        int intA = 10;
        int intB = 20;
        System.out.println("Two integers: " + intA + " + " + intB + " = " + c.add(intA, intB));
        System.out.println("Number of calculations: " + c.calc_count);

        float floatX = 12.5f;
        float floatY = 3.75f;
        System.out.println("Two floats: " + floatX + " + " + floatY + " = " + c.add(floatX, floatY));
        System.out.println("Number of calculations: " + c.calc_count);

        int i1 = 1, i2 = 2, i3 = 3;
        System.out.println("Three integers: " + i1 + " + " + i2 + " + " + i3 + " = " + c.add(i1, i2, i3));
        System.out.println("Number of calculations: " + c.calc_count);
    }
}
```

2. Develop a **Restaurant Billing Application** where overloaded methods calculate bills for dine-in, takeaway, and delivery orders, while static variables track total orders.

Restaurant_Billing.java

Something > src > lab > practice > Restaurant_Billing.java > Restaurant_Billing

```java
package lab.practice;

public class Restaurant_Billing {

    // Static variable to track total orders
    static int total_orders = 0;

    // Dine-in order (5% service charge)
    void calculate_bill(double amount) {
        total_orders++;
        double bill = amount + (amount * 0.05);
        System.out.println("Dine-in Bill = ₹" + bill);
    }

    // Takeaway order (no extra charge)
    void calculate_bill(double amount, boolean takeaway) {
        total_orders++;
        System.out.println("Takeaway Bill = ₹" + amount);
    }

    // Delivery order (fixed delivery charge)
    void calculate_bill(double amount, double deliveryCharge) {
        total_orders++;
        double bill = amount + deliveryCharge;
        System.out.println("Delivery Bill = ₹" + bill);
    }

    Run | Debug
    public static void main(String[] args) {

        Restaurant_Billing order = new Restaurant_Billing();

        // Dine-in
        order.calculate_bill(amount: 800);

        // Takeaway
        order.calculate_bill(amount: 500, takeaway: true);

        // Delivery
        order.calculate_bill(amount: 700, deliveryCharge: 50);

        // Display total orders
        System.out.println("Total Orders = " + total_orders);
    }
}
```